

# LAB – 4

**Project Title:** Adaptive-Software-Development-Lab

**Project Scenario:** Online Course Registration System (OCRS)

## PHASE 1: UNDERSTANDING ADAPTIVE SOFTWARE DEVELOPMENT

### Step 1: Study ASD Characteristics

#### 1. Mission-Focused Development

Mission-focused development means the team concentrates on achieving the main goal of the project instead of strictly following a fixed plan. The focus is on delivering value to users. This allows the team to adjust tasks and priorities if requirements change.

#### 2. Component-Based Development

In ASD, software is developed in small components or modules. Each component performs a specific function and can be developed independently. This approach makes the system flexible and allows easy modification or improvement.

#### 3. Time-Boxed Iterations

ASD uses fixed time periods called iterations to complete tasks. Each iteration has a limited duration, which encourages teams to deliver results quickly. Time-boxing helps maintain progress even when requirements change.

#### 4. Risk-Driven and Change-Tolerant Planning

ASD accepts that changes and risks are part of software development. Instead of avoiding them, teams plan in a way that allows easy adaptation. This helps the project respond effectively to new information and unexpected challenges.

#### 5. Continuous Learning

Continuous learning is an important part of ASD. After each iteration, the team reviews the results and learns from successes and mistakes. This learning helps improve future planning and development decisions.

### Why Traditional Planning Fails in Uncertain Environments

Traditional planning assumes that all requirements are known at the beginning of the project. However, in real software projects, requirements often change due to new technology, user needs, or market conditions. When plans are fixed and rigid, teams cannot easily adapt to these changes. As a result, traditional planning may lead to delays, increased costs, or products that do not meet user expectations.

## Step 2: Understand the SCL Cycle

### 1. Speculate

In a real-world project like an online shopping application, the development team may assume that users prefer quick checkout and multiple payment options. Based on this assumption, they plan features such as guest checkout and digital payments.

### 2. Collaborate

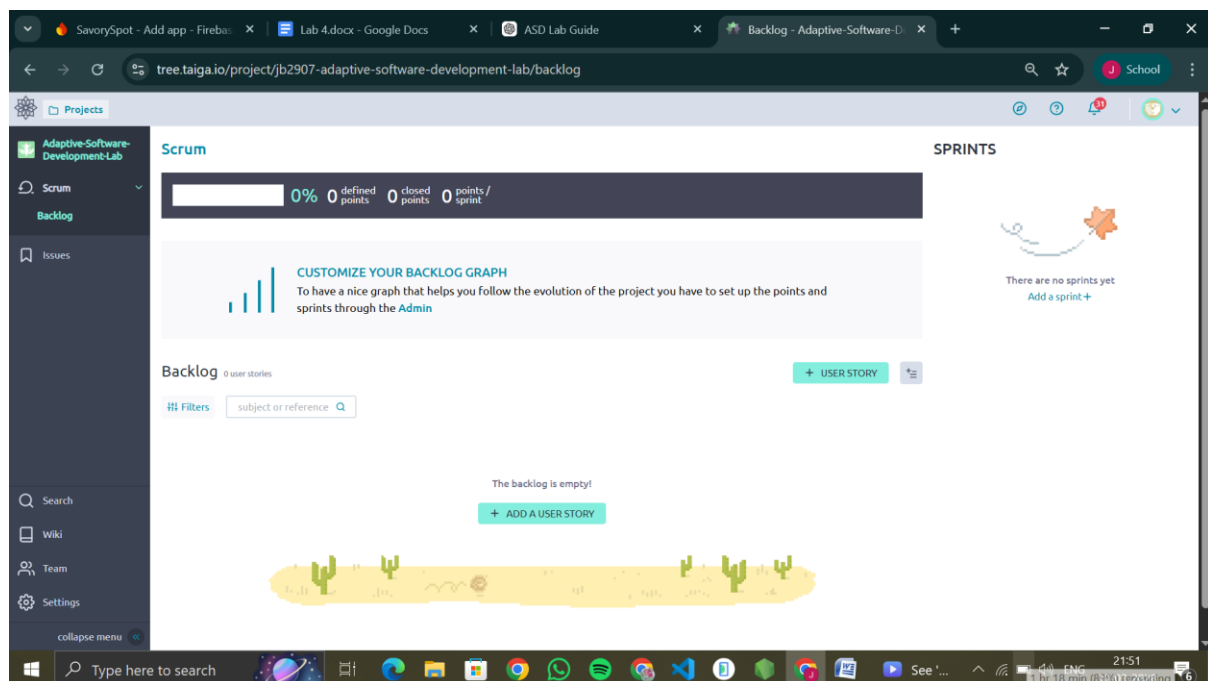
During development, developers, designers, testers, and stakeholders work together to discuss features and solve problems. For example, the development team collaborates with business managers to decide how the product search feature should work.

### 3. Learn

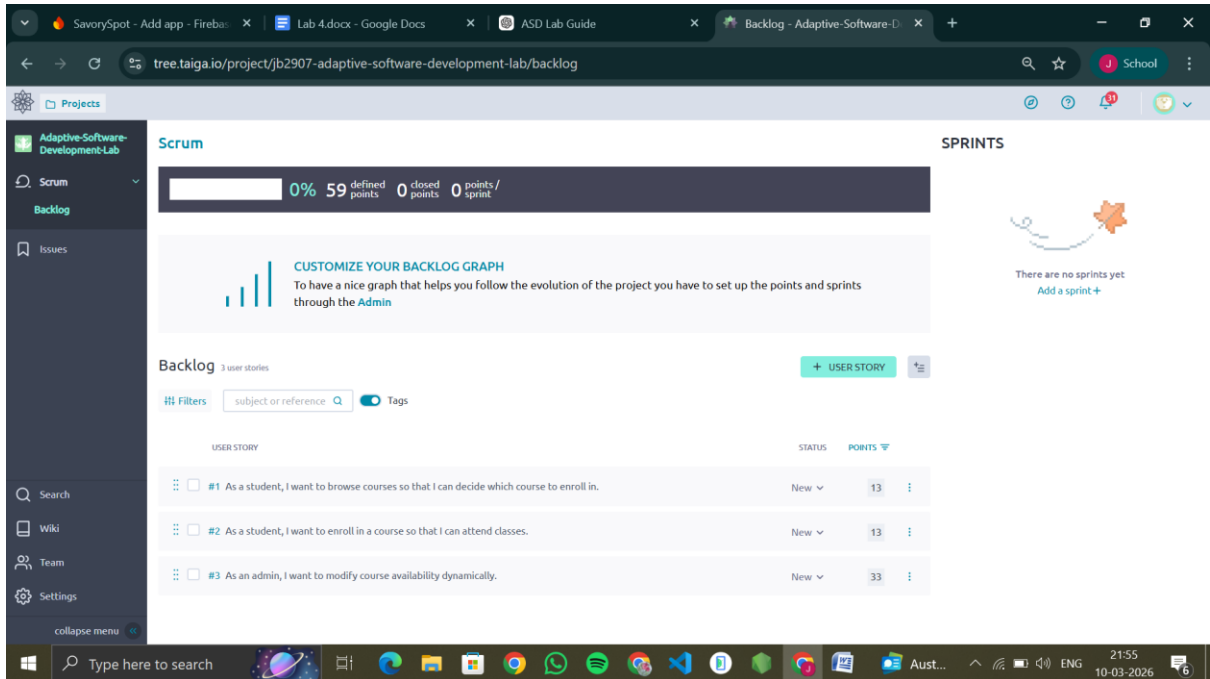
After releasing a feature, the team collects feedback from users. For example, if users report difficulty in using the payment page, the team learns from this feedback and improves the design in the next iteration.

## PHASE 2: SPECULATE PHASE USING TAIGA

### Step 3: Create ASD Project in Taiga



## Step 4: Speculative Planning



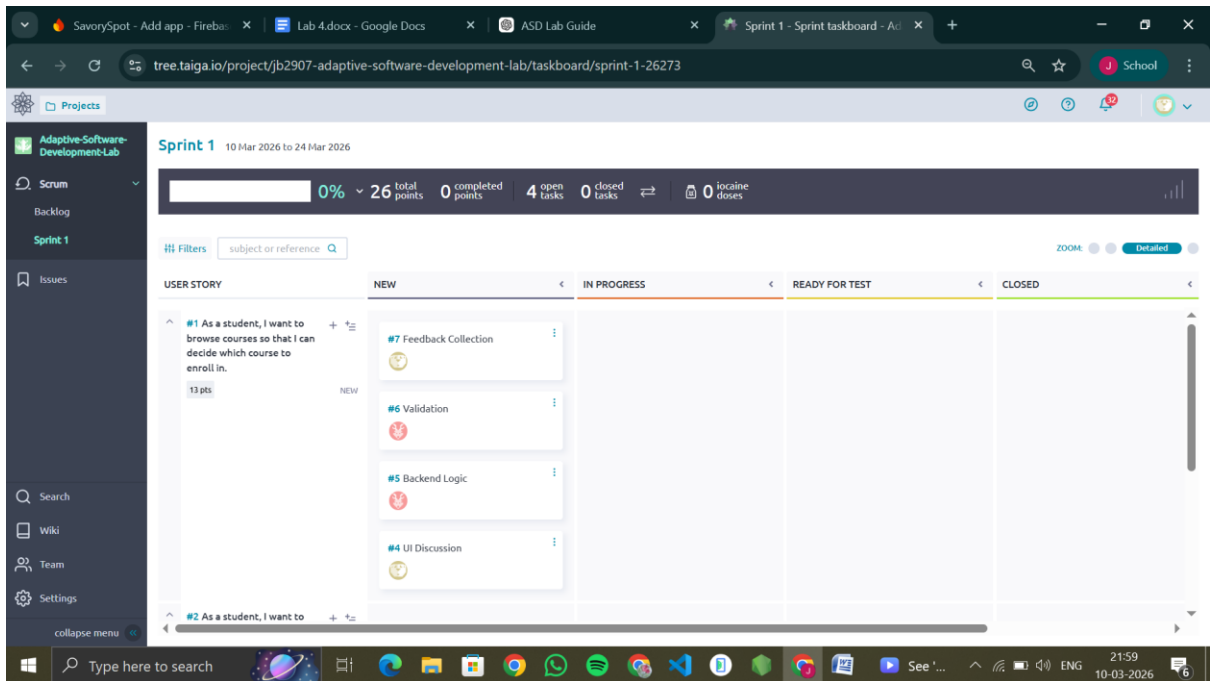
The screenshot shows the Taiga web interface for a project named 'Adaptive-Software-Development-Lab'. The left sidebar contains navigation links for Projects, Scrum, Backlog, and Issues. The main area is titled 'Scrum' and displays a progress bar at 0% with 59 defined points, 0 closed points, and 0 points per sprint. Below this is a section for 'CUSTOMIZE YOUR BACKLOG GRAPH' with a link to 'Admin'. The 'Backlog' section shows 3 user stories with a search filter and a 'Tags' toggle. The user stories are:

| USER STORY                                                                                | STATUS | POINTS |
|-------------------------------------------------------------------------------------------|--------|--------|
| #1 As a student, I want to browse courses so that I can decide which course to enroll in. | New    | 13     |
| #2 As a student, I want to enroll in a course so that I can attend classes.               | New    | 13     |
| #3 As an admin, I want to modify course availability dynamically.                         | New    | 33     |

The right sidebar shows 'SPRINTS' with a message 'There are no sprints yet' and a link to 'Add a sprint+'. The bottom of the screen shows a Windows taskbar with various application icons and a system clock indicating 21:55 on 10-03-2026.

## PHASE 3: COLLABORATE PHASE USING TAIGA

### Step 5: Sprint Creation and Task Collaboration

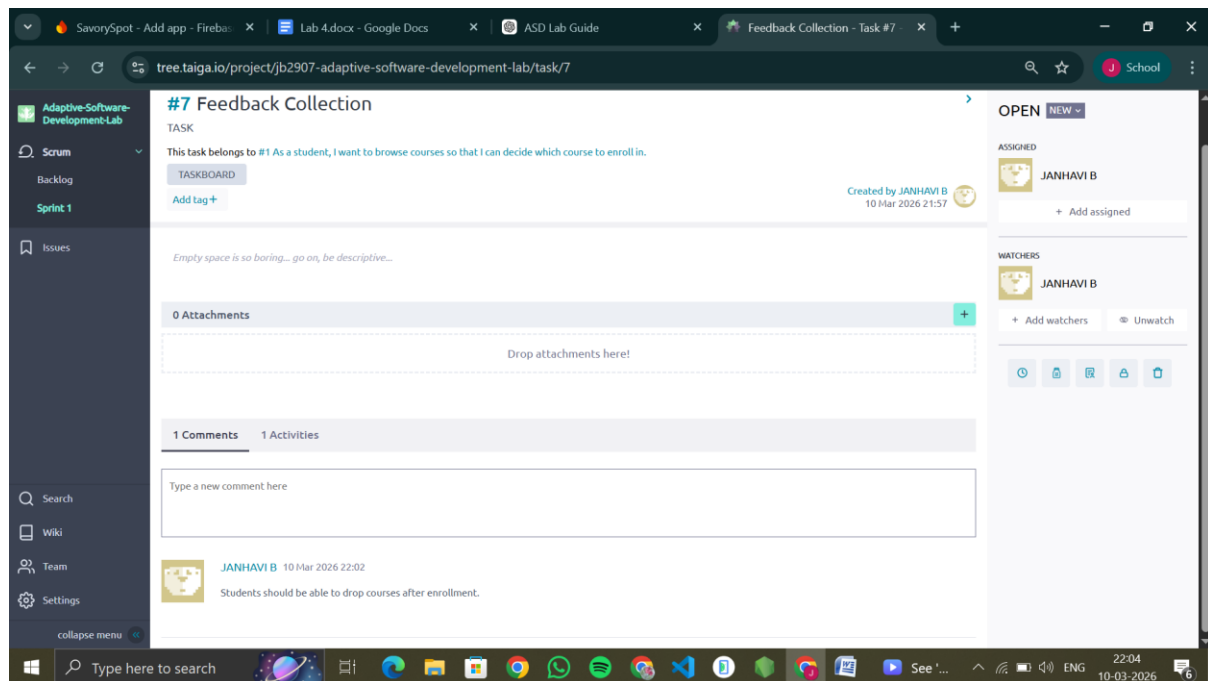


The screenshot shows the Taiga web interface for a specific sprint named 'Sprint 1', which runs from 10 Mar 2026 to 24 Mar 2026. The left sidebar is the same as in the previous screenshot. The main area displays the sprint progress at 0% with 26 total points, 0 completed points, 4 open tasks, 0 closed tasks, and 0 inactive closes. Below this is a Kanban board with columns for 'NEW', 'IN PROGRESS', 'READY FOR TEST', and 'CLOSED'. The 'NEW' column contains four tasks:

- #7 Feedback Collection
- #6 Validation
- #5 Backend Logic
- #4 UI Discussion

The 'IN PROGRESS' column is currently empty. The 'READY FOR TEST' and 'CLOSED' columns are also empty. The bottom of the screen shows a Windows taskbar with various application icons and a system clock indicating 21:59 on 10-03-2026.

## Step 6: Stakeholder Interaction Simulation



## PHASE 4: LEARN PHASE & ADAPTATION

### Step 7: Learning Review

#### 1. Correct Assumptions

- Students want to **browse courses before enrolling**.
- **Online course enrollment** is an important feature.
- **Admins need the ability to update course availability**.

#### 2. Wrong Assumptions

- Initially we assumed **students would not need to cancel or drop courses**.
- We also assumed **payment would be the main concern**, but flexibility in enrollment became more important.

#### 3. New Information Discovered

- Students want the **option to drop courses after enrollment**.
- Users prefer **easy navigation and filtering of courses**.
- Feedback from stakeholders helped **identify new requirements**.

## Step 8: Adaptation & Re-Speculation

### Changes Made to Backlog

- Added a new user story: **Drop Course after Enrollment**.
- Increased priority of **Course Enrollment** and **Browse Courses** features.

### Modified User Stories

- Updated **Course Enrollment** story to include drop option.
- Improved **Browse Courses** story by adding filtering features.

### Plan for Next Iteration

- Implement **Drop Course** feature in the next sprint.
- Improve **course search and filtering system**.
- Collect more **user feedback for further improvements**.